Ex.No.1 - Basic image processing - Converting image to a different colour models

Name: B ELUMALAI

Roll No:230701084

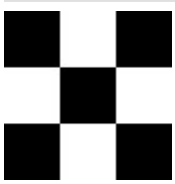
In [2]:
```
import os
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import Image

%matplotlib inline
```

In [3]:
```
# Displaying 18x18 pixel image.
Image(filename = r"D:\VI\IP & CV\Lab Experiments\Inputs\chessboard 18x18.png")
```

Out[3]:

In [4]:
```
# Displaying 84x84 pixel image.
Image(filename = r"D:\VI\IP & CV\Lab Experiments\Inputs\chessboard 84x84.jpg")
```

Out[4]:



In [5]:
```
# Reading image as gray scale.
path = r"D:\VI\IP & CV\Lab Experiments\Inputs\chessboard 18x18.png"
img = cv.imread(path, cv.IMREAD_GRAYSCALE)

# Print the image data (pixel values), element of a 2D numpy array. Each pixel
print(img)
```

```
[[  0   0   0   0   0   0 255 255 255 255 255 255   0   0   0   0   0   0]
 [  0   0   0   0   0   0 255 255 255 255 255 255   0   0   0   0   0   0]
 [  0   0   0   0   0   0 255 255 255 255 255 255   0   0   0   0   0   0]
 [  0   0   0   0   0   0 255 255 255 255 255 255   0   0   0   0   0   0]
 [  0   0   0   0   0   0 255 255 255 255 255 255   0   0   0   0   0   0]
 [  0   0   0   0   0   0 255 255 255 255 255 255   0   0   0   0   0   0]
 [255 255 255 255 255 255   0   0   0   0   0   0 255 255 255 255 255 255]
 [255 255 255 255 255 255   0   0   0   0   0   0 255 255 255 255 255 255]
 [255 255 255 255 255 255   0   0   0   0   0   0 255 255 255 255 255 255]
 [255 255 255 255 255 255   0   0   0   0   0   0 255 255 255 255 255 255]
 [255 255 255 255 255 255   0   0   0   0   0   0 255 255 255 255 255 255]
 [255 255 255 255 255 255   0   0   0   0   0   0 255 255 255 255 255 255]
 [  0   0   0   0   0   0 255 255 255 255 255 255   0   0   0   0   0   0]
 [  0   0   0   0   0   0 255 255 255 255 255 255   0   0   0   0   0   0]
 [  0   0   0   0   0   0 255 255 255 255 255 255   0   0   0   0   0   0]
 [  0   0   0   0   0   0 255 255 255 255 255 255   0   0   0   0   0   0]
 [  0   0   0   0   0   0 255 255 255 255 255 255   0   0   0   0   0   0]
 [  0   0   0   0   0   0 255 255 255 255 255 255   0   0   0   0   0   0]]
```

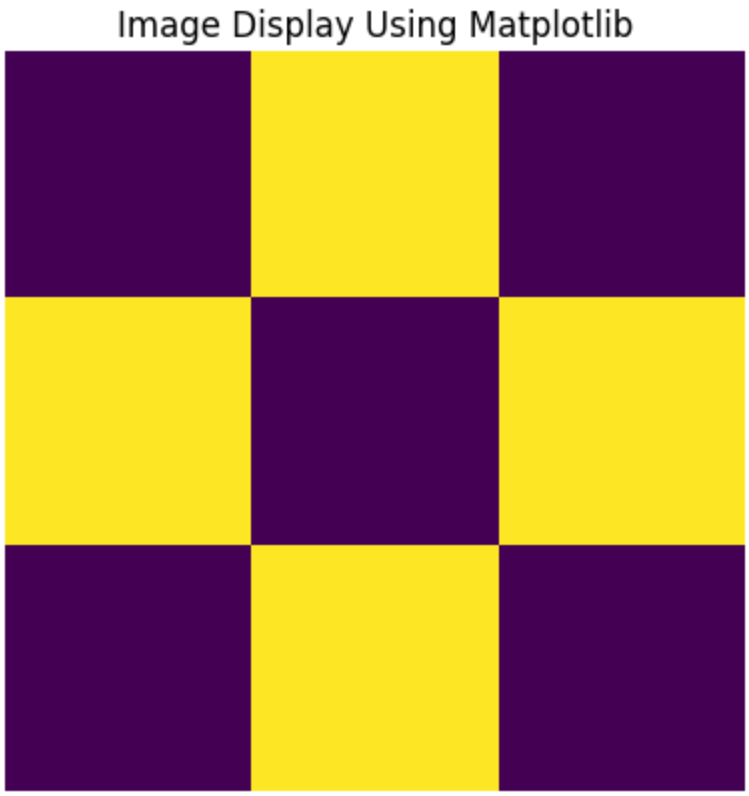
In [6]:

```python
# Displaying the image attributes
print("Shape of the image: ", img.shape)
print("Data type of the image: ", img.dtype)
print("Size of the image: ", img.size)
```

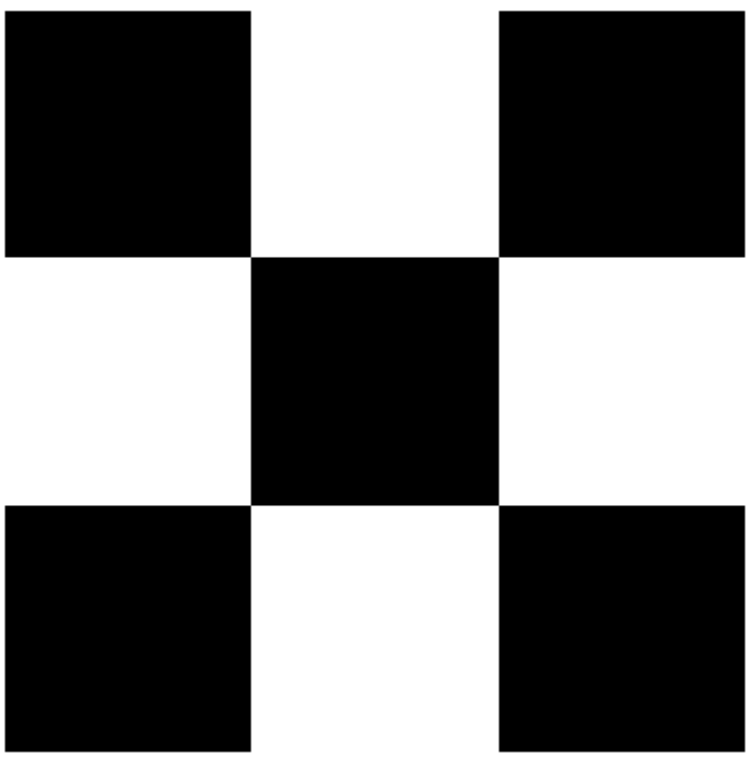
```
Shape of the image:  (18, 18)
Data type of the image:  uint8
Size of the image:  324
```

In [7]:

```python
# Displaying an image using Matplotlib.
plt.imshow(img)
plt.title("Image Display Using Matplotlib")
plt.axis("off") # To turn off axes
plt.show()
```



In [8]:
```
# Set color map to gray scale for proper rendering.
plt.imshow(img, cmap = "gray")
plt.axis("off")
plt.show()
```



In [13]:
```
path2 = r"D:\VI\IP & CV\Lab Experiments\Inputs\doremon.jpg"
Image(filename = path2)
```

Out[13]:

In [14]:
```
#Reading image in BGR
img = cv.imread(path2, cv.IMREAD_COLOR_BGR)

print("Shape of the image: ", img.shape)
print("Data type of the image: ", img.dtype)
print("Size of the image: ", img.size)
```

```
Shape of the image:  (589, 480, 3)
Data type of the image:  uint8
Size of the image:  848160
```

In [15]:
```
#Display the image using matplotlib

plt.imshow(img)
plt.axis("off")
```

```
plt.show()
```

In [16]:
```
#Reversing the channels
reversed_img = img[:, :, ::-1]
plt.imshow(reversed_img)
plt.axis("off")
plt.show()
```

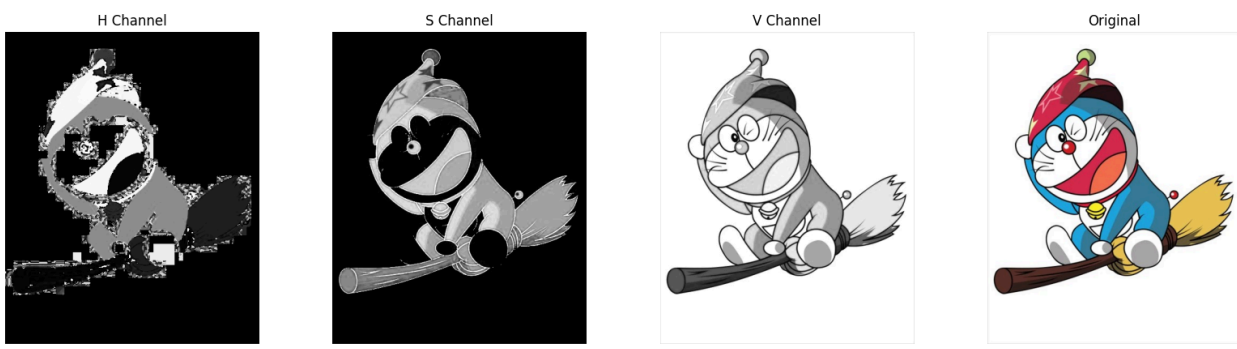
In [17]:
```
# Split the image into the B, G, R components
b, g, r = cv.split(img)

# Show the channels
plt.figure(figsize = [20, 5])

# Red Channel
plt.subplot(1, 4, 1)
plt.imshow(r, cmap = "gray")
plt.title("Red Channel")
plt.axis("off")

# Green Channel
plt.subplot(1, 4, 2)
plt.imshow(g, cmap = "gray")
plt.title("Green Channel")
plt.axis("off")

# Blue Channel
plt.subplot(1, 4, 3)
plt.imshow(b, cmap = "gray")
plt.title("Blue Channel")
plt.axis("off")

# Merge the individual channels into a BGR image
merged_img = cv.merge((b, g, r))

# Show the merged output
plt.subplot(1, 4, 4)
plt.imshow(merged_img[:, :, ::-1])
```

```
plt.title("Merged Output")
plt.axis("off")
plt.show()
```

In [18]:
```
# OpenCV stores color channels in a different order than most other applicatic
img_bgr = cv.imread(path2, cv.IMREAD_COLOR_BGR)

# Converting BGR to RGB
img_rgb = cv.cvtColor(img_bgr, cv.COLOR_BGR2RGB)
plt.imshow(img_rgb)
plt.title("BGR to RGB")
plt.axis("off")
plt.show()
```

In [19]:
```
# Converting to HSV
img_hsv = cv.cvtColor(img_bgr, cv.COLOR_BGR2HSV)

# Split the image into H, S, V components
```

```
h, s, v = cv.split(img_hsv)

# Show the channels
plt.figure(figsize = [20, 5])

# Hue Channel
plt.subplot(1, 4, 1)
plt.imshow(h, cmap = "gray")
plt.title("H Channel")
plt.axis("off")

# Saturation Channel
plt.subplot(1, 4, 2)
plt.imshow(s, cmap = "gray")
plt.title("S Channel")
plt.axis("off")

# Value Channel
plt.subplot(1, 4, 3)
plt.imshow(v, cmap = "gray")
plt.title("V Channel")
plt.axis("off")

# Show the original image
plt.subplot(1, 4, 4)
plt.imshow(img_rgb)
plt.title("Original")
plt.axis("off")
plt.show()
```



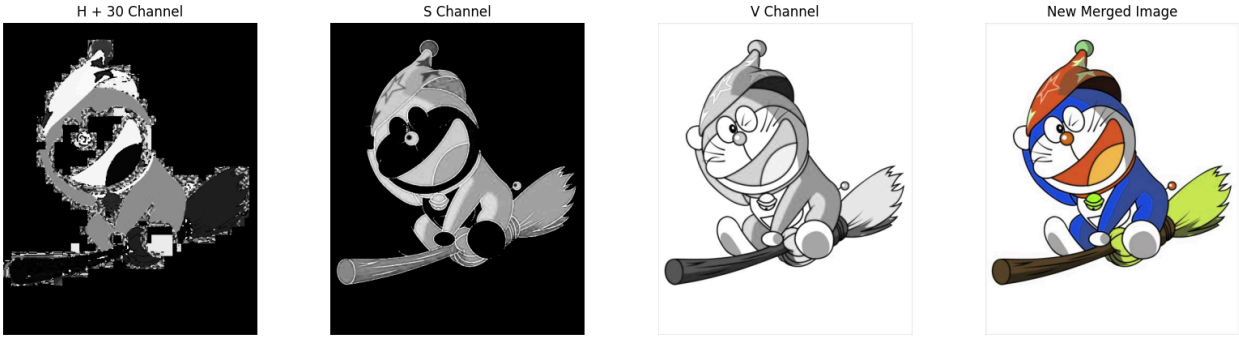
In [20]:

```
# Converting BGR to Grayscale

img_gray = cv.cvtColor(img_bgr, cv.COLOR_BGR2GRAY)
plt.imshow(img_gray, cmap = "gray")
plt.title("BGR to Grayscale")
plt.axis("off")
plt.show()
```

BGR to Grayscale

In [35]:
```
h_new = h + 30
img_merged = cv.merge((h_new, s, v))
img_rgb = cv.cvtColor(img_merged, cv.COLOR_HSV2RGB)

# Split the image into H, S, V components
h, s, v = cv.split(img_merged)

# Show the channels
plt.figure(figsize = [20, 5])

# Hue Channel
plt.subplot(1, 4, 1)
plt.imshow(h, cmap = "gray")
plt.title("H + 30 Channel")
plt.axis("off")

# Saturation Channel
plt.subplot(1, 4, 2)
plt.imshow(s, cmap = "gray")
plt.title("S Channel")
plt.axis("off")

# Value Channel
plt.subplot(1, 4, 3)
plt.imshow(v, cmap = "gray")
plt.title("V Channel")
plt.axis("off")
```

```
# Show the original image
plt.subplot(1, 4, 4)
plt.imshow(img_rgb)
plt.title("New Merged Image")
plt.axis("off")
plt.show()
```



In [36]:
```
# Saving the image
cv.imwrite(r"D:\VI\IP & CV\Lab Experiments\Outputs\doremon_rgb.jpg",img_rgb)
```

Out[36]: True